Kubernetes CI/CD Practical Guide

All 6 practical questions • Python • GitHub Actions • Docker Hub • Kubernetes

Overview

Each practical uses the five-stage pipeline: (1) Source Code, (2) Version Control, (3) Continuous Integration, (4) Build, and (5) Deployment. GitHub Actions checks out the source, installs dependencies, runs tests, builds and pushes a uniquely tagged Docker image, and deploys that exact image to Kubernetes.

Important: make exactly two source-code pushes per practical. Push 1 contains the initial feature set; Push 2 contains the requested additional feature. Use a separate GitHub repository per practical or reset/reconfigure the pipeline carefully when switching projects.

Common project structure

```
project-folder/
├── app.py
├── test_app.py
├── requirements.txt
├── Dockerfile
├── deployment.yaml
├── service.yaml
└── .github/
    └── workflows/
        └── cicd.yml
```

Common files (use for each practical)

requirements.txt

```
Flask>=3.1,<4.0
pytest>=8.0,<10.0
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "-u", "app.py"]
```

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: python-practical
spec:
  replicas: 2
  selector:
    matchLabels:
      app: python-practical
  template:
    metadata:
      labels:
        app: python-practical
    spec:
```

```
      containers:
        - name: python-app
          image: YOUR_DOCKERHUB_USERNAME/python-practical:bootstrap
          ports:
            - containerPort: 5000
          imagePullPolicy: Always
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: python-practical-service
spec:
  type: NodePort
  selector:
    app: python-practical
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
```

.github/workflows/cicd.yml

```
name: Python Kubernetes CI-CD

on:
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run automated tests
        run: python -m pytest -v

      - name: Set unique image tag
        run: echo "IMAGE_TAG=${GITHUB_SHA::12}" >> "$GITHUB_ENV"

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
```

```
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/python-practical:${{ env.IMAGE_TAG }}

      - name: Configure kubectl
        env:
          KUBE_CONFIG: ${{ secrets.KUBE_CONFIG }}
        run: |
          mkdir -p "$HOME/.kube"
          echo "$KUBE_CONFIG" | base64 -d > "$HOME/.kube/config"
          chmod 600 "$HOME/.kube/config"
          kubectl cluster-info

      - name: Apply Kubernetes manifests
        run: |
          kubectl apply -f deployment.yaml
          kubectl apply -f service.yaml

      - name: Deploy the exact image built by this push
        run: |
          kubectl set image deployment/python-practical             python-app=${{ secrets.DOCKERHUB_USERNAME
}}/python-practical:${{ env.IMAGE_TAG }}
          kubectl rollout status deployment/python-practical --timeout=180s
```

Practical 1 — Menu-Driven Calculator

Push 1 — Initial application

app.py

```
from flask import Flask, request
app = Flask(__name__)

def calculate(a, b, operation):
    if operation == "add":
        return a + b
    if operation == "subtract":
        return a - b
    raise ValueError("Unsupported operation")

@app.route("/")
def home():
    return '''
    <h1>Menu-Driven Calculator</h1>
    <form action="/calculate" method="post">
      <input name="a" type="number" step="any" placeholder="First number" required>
      <input name="b" type="number" step="any" placeholder="Second number" required>
      <select name="operation">
        <option value="add">Addition</option>
        <option value="subtract">Subtraction</option>
      </select>
      <button>Calculate</button>
    </form>'''

@app.route("/calculate", methods=["POST"])
def calculate_route():
    try:
        result = calculate(float(request.form["a"]), float(request.form["b"]),
                           request.form["operation"])
        return f"<h2>Result: {result}</h2><a href='/'>Back</a>"
    except (ValueError, KeyError) as error:
        return str(error), 400

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

test_app.py

```
from app import app, calculate
import pytest

def test_addition():
    assert calculate(10, 5, "add") == 15

def test_subtraction():
    assert calculate(10, 5, "subtract") == 5

def test_invalid_operation():
    with pytest.raises(ValueError):
        calculate(10, 5, "multiply")

def test_home_page():
    assert app.test_client().get("/").status_code == 200
```

Push 2 — Required modification

For Push 2, replace calculate() with:

```
def calculate(a, b, operation):
    if operation == "add": return a + b
```

```
    if operation == "subtract": return a - b
    if operation == "multiply": return a * b
    if operation == "divide":
        if b == 0: raise ValueError("Cannot divide by zero")
        return a / b
    raise ValueError("Unsupported operation")
```

Add multiply and divide options to the HTML <select> menu.
Add tests:

```
def test_multiplication():
    assert calculate(4, 3, "multiply") == 12
def test_division():
    assert calculate(12, 3, "divide") == 4
def test_division_by_zero():
    with pytest.raises(ValueError):
        calculate(10, 0, "divide")
```

Practical 2 — Student Grade Calculator

Push 1 — Initial application

app.py

```
from flask import Flask, request
app = Flask(__name__)

def calculate_marks(marks):
    if not marks or any(m < 0 or m > 100 for m in marks):
        raise ValueError("Marks must be between 0 and 100")
    total = sum(marks)
    average = total / len(marks)
    return total, average

@app.route("/")
def home():
    return '''
    <h1>Student Grade Calculator</h1>
    <p>Enter marks separated by commas, e.g. 80,75,90</p>
    <form action="/calculate" method="post">
      <input name="marks" placeholder="80,75,90" required>
      <button>Calculate</button>
    </form>'''

@app.route("/calculate", methods=["POST"])
def calculate_route():
    try:
        marks = [float(x.strip()) for x in request.form["marks"].split(",")]
        total, average = calculate_marks(marks)
        return f"<h2>Total: {total}</h2><h2>Average: {average:.2f}</h2>"
    except ValueError as error:
        return str(error), 400

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

test_app.py

```
from app import app, calculate_marks
import pytest

def test_total_and_average():
    assert calculate_marks([80, 70, 90]) == (240, 80)

def test_invalid_marks():
    with pytest.raises(ValueError):
        calculate_marks([120, 50])

def test_home():
    assert app.test_client().get("/").status_code == 200
```

Push 2 — Required modification

For Push 2, replace calculate_marks() with:

```
def calculate_marks(marks):
    if not marks or any(m < 0 or m > 100 for m in marks):
        raise ValueError("Marks must be between 0 and 100")
    total = sum(marks)
    average = total / len(marks)
    if average >= 90: grade = "A"
    elif average >= 80: grade = "B"
    elif average >= 70: grade = "C"
```

```
    elif average >= 60: grade = "D"
    else: grade = "F"
    return total, average, grade
```

Update route to unpack total, average, grade and display all three.
Update tests:

```
def test_grade_calculation():
    assert calculate_marks([95, 85, 90]) == (270, 90, "A")
```

These are example grade thresholds; use your instructor's rules if supplied.

Practical 3 — Shopping Bill Calculator

Push 1 — Initial application

app.py

```
from flask import Flask, request
app = Flask(__name__)

def calculate_bill(price, quantity):
    if price < 0 or quantity <= 0:
        raise ValueError("Price must be non-negative and quantity positive")
    return price * quantity

@app.route("/")
def home():
    return '''
    <h1>Shopping Bill Calculator</h1>
    <form action="/bill" method="post">
      <input name="price" type="number" min="0" step="any" placeholder="Item price" required>
      <input name="quantity" type="number" min="1" placeholder="Quantity" required>
      <button>Calculate Bill</button>
    </form>'''

@app.route("/bill", methods=["POST"])
def bill_route():
    try:
        total = calculate_bill(float(request.form["price"]), int(request.form["quantity"]))
        return f"<h2>Total bill: ₹{total:.2f}</h2>"
    except ValueError as error:
        return str(error), 400

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

test_app.py

```
from app import app, calculate_bill
import pytest

def test_total_bill():
    assert calculate_bill(50, 3) == 150

def test_zero_quantity():
    with pytest.raises(ValueError):
        calculate_bill(50, 0)

def test_home():
    assert app.test_client().get("/").status_code == 200
```

Push 2 — Required modification

For Push 2, replace calculate_bill() with:

```
def calculate_bill(price, quantity, discount=0):
  if price < 0 or quantity <= 0:
    raise ValueError("Invalid price or quantity")
  if not 0 <= discount <= 100:
    raise ValueError("Discount must be between 0 and 100")
  total = price * quantity
  return total, total * (1 - discount / 100)
```

Add a discount percentage input to the form. Update the route to read request.form.get("discount", 0), unpack total and final_total, and display both.

Add test:

```
def test_discount():
    assert calculate_bill(100, 2, 10) == (200, 180)
```

Practical 4 — Temperature Converter

Push 1 — Initial application

app.py

```
from flask import Flask, request
app = Flask(__name__)

def convert_temperature(value, direction):
    if direction == "c_to_f":
        return (value * 9 / 5) + 32
    raise ValueError("Unsupported conversion")

@app.route("/")
def home():
    return '''
    <h1>Temperature Converter</h1>
    <form action="/convert" method="post">
      <input name="temperature" type="number" step="any" placeholder="Temperature in Celsius" required>
      <button>Convert to Fahrenheit</button>
    </form>'''

@app.route("/convert", methods=["POST"])
def convert_route():
    try:
        result = convert_temperature(float(request.form["temperature"]), "c_to_f")
        return f"<h2>Temperature: {result:.2f} °F</h2>"
    except ValueError as error:
        return str(error), 400

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

test_app.py

```
from app import app, convert_temperature
import pytest

def test_celsius_to_fahrenheit():
    assert convert_temperature(0, "c_to_f") == 32

def test_unsupported_conversion():
    with pytest.raises(ValueError):
        convert_temperature(32, "f_to_c")

def test_home():
    assert app.test_client().get("/").status_code == 200
```

Push 2 — Required modification

For Push 2, replace convert_temperature() with:

```
def convert_temperature(value, direction):
  if direction == "c_to_f": return (value * 9 / 5) + 32
  if direction == "f_to_c": return (value - 32) * 5 / 9
  raise ValueError("Unsupported conversion")
```

Add radio buttons to the form with values c_to_f and f_to_c, and update the route to pass request.form["direction"].

Add test:

```
def test_fahrenheit_to_celsius():
  assert convert_temperature(32, "f_to_c") == 0
```

Practical 5 — Simple Banking Application

Push 1 — Initial application

app.py

```
from flask import Flask, request
app = Flask(__name__)
balance = 1000.0

def deposit(amount):
    global balance
    if amount <= 0:
        raise ValueError("Deposit must be positive")
    balance += amount
    return balance

@app.route("/")
def home():
    return '''
    <h1>Simple Banking Application</h1>
    <form action="/deposit" method="post">
      <input name="amount" type="number" min="0.01" step="any" placeholder="Deposit amount" required>
      <button>Deposit</button>
    </form>
    <a href="/balance">Check balance</a>'''

@app.route("/balance")
def check_balance():
    return f"<h2>Current balance: ₹{balance:.2f}</h2>"

@app.route("/deposit", methods=["POST"])
def deposit_route():
    try:
        new_balance = deposit(float(request.form["amount"]))
        return f"<h2>New balance: ₹{new_balance:.2f}</h2>"
    except ValueError as error:
        return str(error), 400

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

test_app.py

```
import pytest
import app as banking

def setup_function():
    banking.balance = 1000.0

def test_deposit():
    assert banking.deposit(200) == 1200

def test_invalid_deposit():
    with pytest.raises(ValueError):
        banking.deposit(-10)

def test_balance_page():
    assert banking.app.test_client().get("/balance").status_code == 200
```

Push 2 — Required modification

For Push 2, add:

def withdraw(amount):

  global balance

```
    if amount <= 0: raise ValueError("Withdrawal must be positive")
    if amount > balance: raise ValueError("Insufficient balance")
    balance -= amount
    return balance
```

Add a /withdraw POST route and a withdrawal form to the home page.
Add tests:

```
def test_withdrawal():
    assert banking.withdraw(300) == 700
def test_insufficient_balance():
    with pytest.raises(ValueError):
        banking.withdraw(2000)
```

This is an in-memory educational demo; the balance resets on restart.

Practical 6 — Library Book Management

Push 1 — Initial application

app.py

```
from flask import Flask, request
app = Flask(__name__)
books = []

def add_book(title):
    title = title.strip()
    if not title:
        raise ValueError("Book title cannot be empty")
    books.append({"title": title, "available": True})
    return books

@app.route("/")
def home():
    return '''
    <h1>Library Book Management</h1>
    <form action="/add" method="post">
      <input name="title" placeholder="Book title" required>
      <button>Add Book</button>
    </form>
    <a href="/books">View books</a>'''

@app.route("/add", methods=["POST"])
def add_route():
    try:
        add_book(request.form["title"])
        return "<h2>Book added successfully</h2><a href='/'>Back</a>"
    except ValueError as error:
        return str(error), 400

@app.route("/books")
def view_books():
    if not books:
        return "<h2>No books available</h2>"
    return "<h1>Books</h1><ul>" + "".join(
        f"<li>{book['title']} — Available</li>" for book in books
    ) + "</ul>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

test_app.py

```
import pytest
import app as library

def setup_function():
    library.books.clear()

def test_add_book():
    library.add_book("Python Basics")
    assert len(library.books) == 1

def test_empty_title():
    with pytest.raises(ValueError):
        library.add_book("")

def test_view_books():
    assert library.app.test_client().get("/books").status_code == 200
```

Push 2 — Required modification

For Push 2, add:

```
def issue_book(title):
    for book in books:
        if book["title"].lower() == title.lower():
            if not book["available"]:
                raise ValueError("Book is already issued")
            book["available"] = False
            return book
    raise ValueError("Book not found")
```

Add an /issue POST route and issue form to the home page.
Update /books to display Available or Issued based on book["available"].
Add tests:

```
def test_issue_book():
    library.add_book("Python Basics")
    assert library.issue_book("Python Basics")["available"] is False
def test_issue_same_book_twice():
    library.add_book("Python Basics")
    library.issue_book("Python Basics")
    with pytest.raises(ValueError):
        library.issue_book("Python Basics")
```

Execution and two-push procedure

GitHub repository setup

Create a GitHub repository for the practical and add the following Actions secrets under Settings → Secrets and variables → Actions:

- DOCKERHUB_USERNAME — your Docker Hub username
- DOCKERHUB_TOKEN — a Docker Hub access token
- KUBE_CONFIG — base64-encoded kubeconfig for a Kubernetes cluster reachable from GitHub Actions

Push 1 — run in the VS Code PowerShell terminal

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m pytest -v

git init
git branch -M main
git add .
git commit -m "Push 1: Initial application"
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

Push 2 — after modifying app.py and test_app.py

```
python -m pytest -v
git add .
git commit -m "Push 2: Add requested features"
git push origin main
```

If the repository already exists and has a remote configured, do not repeat git init or git remote add origin. Ensure both pushes contain working tests. A failed test should stop the workflow before Docker image publication and deployment.

Kubernetes verification

```
kubectl get deployments
kubectl get pods
kubectl get services
kubectl rollout status deployment/python-practical
kubectl get deployment python-practical -o jsonpath="{.spec.template.spec.containers[0].image}"
kubectl port-forward service/python-practical-service 8080:80
```

Open http://localhost:8080 while the port-forward command is running.

Critical notes

- The workflow tags each image with the first 12 characters of the Git commit SHA, so each source push receives a unique tag.
- The workflow deploys the exact tag built by that run. The Kubernetes manifest's bootstrap tag is only a placeholder; the workflow's kubectl set image step sets the real tag.
- A GitHub-hosted runner normally cannot reach a Kubernetes cluster running only on your own PC. Use a reachable cluster or a properly configured self-hosted runner. A kubeconfig secret alone does not make a private local cluster reachable.
- If Docker Hub images are private, configure Kubernetes image-pull credentials. Public images avoid that extra setup.
- Use separate repositories or distinct deployment/image names if you want all six practicals to run concurrently. This guide reuses the name python-practical for one practical at a time.
- The examples are educational demos. Banking balance and library books are stored in memory and reset when the Python process restarts.